

CatalogOps — MVP

A rule-based product-catalog audit tool: upload a CSV/Excel export, get a plain-language report on duplicate SKUs, missing fields, inconsistent titles, and broken variant groups — plus a root-cause explanation for each pattern, not just a list of symptoms.

What's actually built here

- `audit_engine.py` — Stage 1 (ingest) + Stage 2 (rule-based checks) + Stage 3 (root-cause grouping). Every check is deterministic and explainable; nothing is silently guessed.
- `report_generator.py` — builds the branded PDF report (same navy/amber identity as the VBA-audit business), reusing that reportlab pipeline.
- `app.py` — the Streamlit MVP UI: upload → run → view findings → download the PDF.
- `sample_catalog.csv` — a realistic test file with intentional issues (a duplicate SKU, missing prices, a missing category, broken variant groups) to demo the tool without needing a real client file yet.

Run it locally

```
bash

pip install -r requirements.txt
streamlit run app.py
```

Then open the local URL Streamlit prints (usually `http://localhost:8501`), and upload `sample_catalog.csv` to see it work end to end.

Deploy for free (per the tech-stack design doc)

1. Push this folder to a GitHub repo (keep it **private** — this is your actual product logic).
2. Go to share.streamlit.io, connect your GitHub account, and point it at this repo's `app.py`.
3. Streamlit Community Cloud builds and hosts it for free. You'll get a public URL you can link from the CatalogOps landing page's upload button.

Known limitations (be upfront about these with early clients)

- **Title-similarity matching isn't perfect.** It correctly ignores legitimate variants (e.g. "Small" vs "Large" of the same product) but will miss some real typo-duplicates that don't share a common word stem (tested case: "Phone Cse Black Edition" vs "Phone Case - Black" wasn't caught). This is a real trade-off in fuzzy string matching, not a bug — tightening the match would reintroduce false positives on legitimate variants.
- **Column detection is alias-based**, not ML-driven. It recognizes common Shopify/Amazon/generic column names. An unusually-named column won't be

picked up automatically — this is a place to expand the `COLUMN_ALIASES` dict as real client files reveal new naming patterns.

- **No live channel integration.** This works from exported files only, per the MVP scope in the PRD — live API sync is a deliberately later feature, not a v1 gap.
- **$O(n^2)$ title comparison** is fine at MVP scale (hundreds to low thousands of rows) but will slow down on very large catalogs — revisit with proper blocking/indexing if that becomes a real constraint.

Next steps, in priority order

1. Run this against a **real** client catalog (even a messy export a friend's e-commerce business can share) to see what column-naming and error patterns the sample file didn't anticipate.
2. Expand `COLUMN_ALIASES` and add new check functions based on what real data reveals — don't guess ahead of real usage.
3. Once there's a paying pilot, revisit the "processed in a shared free-tier environment" trade-off noted in the tech-stack doc.